

LeetCode SSSP Problem Set — Specs, Algorithms & Code

Practice set for CSE208 Lab Test (Single Source Shortest Path)

Prepared for Sajib

July 8, 2026

Contents

Base Template (Reference)	1
1. LeetCode 743 — Network Delay Time (Medium)	3
2. LeetCode 787 — Cheapest Flights Within K Stops (Medium)	4
3. LeetCode 1631 — Path With Minimum Effort (Medium)	5
4. LeetCode 1514 — Path with Maximum Probability (Medium)	7
5. LeetCode 1976 — Number of Ways to Arrive at Destination (Medium)	9
6. LeetCode 2045 — Second Minimum Time to Reach Destination (Hard)	11
Summary Table	13

Base Template (Reference)

Every problem below is a **variation of Dijkstra’s algorithm** — same min-heap greedy structure, different “relax” rule or a different quantity being tracked. Keep this skeleton in your head:

```
struct DijkstraResult {
    vector<long long> dist;
    vector<int> parent;
};

DijkstraResult dijkstra(vector<vector<pair<int, long long>>>& adj, int n, int src) {
    vector<long long> dist(n, LLONG_MAX);
    vector<int> parent(n, -1);
    priority_queue<pair<long long, int>, vector<pair<long long, int>>, greater<pair<long long, int>>> pq;

    dist[src] = 0;
    pq.push({0, src});

    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
```

```

    if (d > dist[u]) continue;           // stale entry, skip

    for (auto& [v, w] : adj[u]) {
        if (dist[u] + w < dist[v]) {     // <-- THIS LINE is what changes per pro
            dist[v] = dist[u] + w;
            parent[v] = u;
            pq.push({dist[v], v});
        }
    }
}
return {dist, parent};
}

```

What changes between problems, almost always: 1. The **relax condition** ($\text{dist}[u] + w < \text{dist}[v]$) — sometimes it's max instead of $+$, sometimes $*$ instead of $+$, sometimes with $>$ instead of $<$. 2. What you **track alongside $\text{dist}[]$** — sometimes a count of ways, sometimes a step/stop limit. 3. Whether you need **Dijkstra at all**, or a **Bellman-Ford-style bounded relaxation** because the constraint (like “at most K edges”) breaks Dijkstra’s greedy assumption.

1. LeetCode 743 — Network Delay Time (Medium)

Specification

- n nodes labeled $1..n$, $times[i] = [u, v, w]$ = directed edge $u \rightarrow v$ taking w time.
- A signal starts at node k . Return the time for the signal to reach **all** nodes, or -1 if some node is unreachable.

Example

Input: $times = [[2,1,1],[2,3,1],[3,4,1]]$, $n = 4$, $k = 2$

Output: 2

Algorithm (plain Dijkstra — no changes to the base template)

1. Build a directed adjacency list.
2. Run Dijkstra from k .
3. Answer = $\max(dist[])$ over all nodes. If any $dist[i] == INF$, return -1 .

Complexity: $O((n + m) \log n)$.

Code

```
class Solution {
public:
    int networkDelayTime(vector<vector<int>>& times, int n, int k) {
        vector<vector<pair<int,int>>> adj(n + 1);
        for (auto& t : times) adj[t[0]].push_back({t[1], t[2]});

        vector<long long> dist(n + 1, LLONG_MAX);
        priority_queue<pair<long long,int>, vector<pair<long long,int>>,
            greater<pair<long long,int>>> pq;

        dist[k] = 0;
        pq.push({0, k});

        while (!pq.empty()) {
            auto [d, u] = pq.top(); pq.pop();
            if (d > dist[u]) continue;
            for (auto& [v, w] : adj[u]) {
                if (dist[u] + w < dist[v]) {
                    dist[v] = dist[u] + w;
                    pq.push({dist[v], v});
                }
            }
        }

        long long ans = 0;
        for (int i = 1; i <= n; i++) {
            if (dist[i] == LLONG_MAX) return -1;
            ans = max(ans, dist[i]);
        }
        return (int)ans;
    }
};
```

2. LeetCode 787 — Cheapest Flights Within K Stops (Medium)

Specification

- n cities, flights[i] = [from, to, price] (directed).
- Find the cheapest price from src to dst using **at most k stops** (i.e. at most k+1 edges). Return -1 if no such route exists.

Example

Input: n = 4, flights = [[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]]
src = 0, dst = 3, k = 1
Output: 700 (path 0 -> 1 -> 3, using exactly 1 stop)

Algorithm (Bellman-Ford style — NOT plain Dijkstra)

This is the one case where the base template's greedy assumption **breaks**: Dijkstra finalizes a node's distance as soon as it's popped, but that "cheapest overall" path might use *more* stops than allowed. So instead:

1. Run **Bellman-Ford limited to exactly k+1 rounds** of relaxation.
2. Each round, relax every edge using distances **from the previous round only** (use a copy of dist[], not update it in place) — this guarantees round i only uses paths with ≤ i edges.
3. After k+1 rounds, dist[dst] is the answer (or -1 if still INF).

Complexity: O(k * m).

Code

```
class Solution {
public:
    int findCheapestPrice(int n, vector<vector<int>>& flights, int src, int dst, int k) {
        vector<long long> dist(n, LLONG_MAX);
        dist[src] = 0;

        for (int round = 0; round <= k; round++) {
            vector<long long> newDist = dist;           // freeze previous round's values
            for (auto& f : flights) {
                int u = f[0], v = f[1], w = f[2];
                if (dist[u] != LLONG_MAX && dist[u] + w < newDist[v]) {
                    newDist[v] = dist[u] + w;
                }
            }
            dist = newDist;
        }
        return dist[dst] == LLONG_MAX ? -1 : (int)dist[dst];
    }
};
```

3. LeetCode 1631 — Path With Minimum Effort (Medium)

Specification

- heights[row][col] grid. You move between 4-directionally adjacent cells.
- The “effort” of a path = the **maximum absolute height difference** between two consecutive cells on that path (not the sum).
- Find a path from top-left to bottom-right that **minimizes** this maximum effort.

Example

Input: heights = [[1,2,2],[3,8,2],[5,3,5]]

Output: 2

Algorithm (Dijkstra, but relax with max instead of +)

This is a **minimax path** problem — exactly one line in the base template changes:

`dist[u] + w < dist[v] --> max(dist[u], w) < dist[v]`

1. Treat the grid as a graph: each cell is a node, edge weight to a neighbor = `abs(height difference)`.
2. Run Dijkstra from (0,0), but the “distance” being relaxed is the **max edge weight seen so far on the path**, not a sum.
3. Answer = `dist[bottom-right]`.

Complexity: $O(\text{rows} * \text{cols} * \log(\text{rows} * \text{cols}))$.

Code

```
class Solution {
public:
    int minimumEffortPath(vector<vector<int>>& heights) {
        int rows = heights.size(), cols = heights[0].size();
        vector<vector<int>> dist(rows, vector<int>(cols, INT_MAX));
        priority_queue<tuple<int,int,int>, vector<tuple<int,int,int>>, greater<>> pq;

        dist[0][0] = 0;
        pq.push({0, 0, 0}); // {effort, row, col}
        int dr[] = {0, 0, 1, -1}, dc[] = {1, -1, 0, 0};

        while (!pq.empty()) {
            auto [d, r, c] = pq.top(); pq.pop();
            if (d > dist[r][c]) continue;
            if (r == rows - 1 && c == cols - 1) return d;

            for (int dir = 0; dir < 4; dir++) {
                int nr = r + dr[dir], nc = c + dc[dir];
                if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;

                int newEffort = max(d, abs(heights[nr][nc] - heights[r][c]));
                if (newEffort < dist[nr][nc]) {
                    dist[nr][nc] = newEffort;
                    pq.push({newEffort, nr, nc});
                }
            }
        }
    }
};
```

```
    }  
  }  
  return 0;  
};
```

4. LeetCode 1514 — Path with Maximum Probability (Medium)

Specification

- n nodes, $\text{edges}[i] = [a, b]$ with $\text{succProb}[i] = \text{probability of successfully traversing that edge (undirected)}$.
- Find the path from start to end that **maximizes** the product of probabilities along it. Return 0 if unreachable.

Example

Input: $n = 3$, $\text{edges} = [[0,1],[1,2],[0,2]]$, $\text{succProb} = [0.5,0.5,0.3]$
 $\text{start} = 0$, $\text{end} = 2$

Output: 0.25 (path $0 \rightarrow 1 \rightarrow 2$: $0.5 * 0.5 = 0.25$, beats direct edge 0.3)

Algorithm (Dijkstra, but maximize a product with a max-heap)

Two changes from the base template:

1. **Relax rule:** $\text{dist}[u] * w > \text{dist}[v]$ (multiply, and look for **larger**, not smaller).
2. **Heap direction:** use a **max-heap** (default `priority_queue`, no `greater<>`), since we always want to expand the currently-most-probable node first — this is the mirror image of normal Dijkstra's "always expand closest node."

Complexity: $O((n + m) \log n)$.

Code

```
class Solution {
public:
    double maxProbability(int n, vector<vector<int>>& edges, vector<double>& succProb,
                          int start, int end) {
        vector<vector<pair<int,double>>> adj(n);
        for (int i = 0; i < (int)edges.size(); i++) {
            int a = edges[i][0], b = edges[i][1];
            adj[a].push_back({b, succProb[i]});
            adj[b].push_back({a, succProb[i]});
        }

        vector<double> dist(n, 0.0);
        dist[start] = 1.0;
        priority_queue<pair<double,int>> pq; // max-heap by default
        pq.push({1.0, start});

        while (!pq.empty()) {
            auto [d, u] = pq.top(); pq.pop();
            if (d < dist[u]) continue; // stale entry
            if (u == end) return d;

            for (auto& [v, w] : adj[u]) {
                if (dist[u] * w > dist[v]) {
                    dist[v] = dist[u] * w;
                    pq.push({dist[v], v});
                }
            }
        }

        return 0;
    }
};
```

```
    }  
    }  
    }  
    return 0.0;  
};
```

5. LeetCode 1976 — Number of Ways to Arrive at Destination (Medium)

Specification

- n intersections, $\text{roads}[i] = [u, v, \text{time}]$ (undirected).
- Count the number of **distinct paths** from node 0 to node $n-1$ that take the minimum possible time. Return the count modulo $1e9 + 7$.

Example

Input: $n = 7$, $\text{roads} = [[0,6,7],[0,1,2],[1,2,3],[1,3,3],[6,3,3],[3,5,1],[6,5,1],[2,5,1],[0,4,4]]$
Output: 4

Algorithm (Dijkstra + a parallel ways[] counter)

Track a second array $\text{ways}[]$ alongside $\text{dist}[]$:

- **Strictly shorter path found** ($\text{dist}[u] + w < \text{dist}[v]$): reset $\text{ways}[v] = \text{ways}[u]$ (previous ways are now obsolete — this route is unambiguously better).
- **Equally short path found** ($\text{dist}[u] + w == \text{dist}[v]$): accumulate $\text{ways}[v] += \text{ways}[u]$ (this is an *additional* way to achieve the same minimum time).

This is the standard “shortest path counting” pattern — same base template, one extra branch.

Complexity: $O((n + m) \log n)$.

Code

```
class Solution {
public:
    int countPaths(int n, vector<vector<int>>& roads) {
        const long long MOD = 1e9 + 7;
        vector<vector<pair<int, long long>>> adj(n);
        for (auto& r : roads) {
            adj[r[0]].push_back({r[1], r[2]});
            adj[r[1]].push_back({r[0], r[2]});
        }

        vector<long long> dist(n, LLONG_MAX), ways(n, 0);
        dist[0] = 0;
        ways[0] = 1;
        priority_queue<pair<long long, int>, vector<pair<long long, int>>,
                      greater<pair<long long, int>>> pq;
        pq.push({0, 0});

        while (!pq.empty()) {
            auto [d, u] = pq.top(); pq.pop();
            if (d > dist[u]) continue;

            for (auto& [v, w] : adj[u]) {
                if (dist[u] + w < dist[v]) {
                    dist[v] = dist[u] + w;
                    ways[v] = ways[u];
                    pq.push({dist[v], v});
                }
            }
        }

        return ways[n-1];
    }
};
```

```

        } else if (dist[u] + w == dist[v]) {
            ways[v] = (ways[v] + ways[u]) % MOD;
        }
    }
}
return (int)(ways[n - 1] % MOD);
}
};

```

6. LeetCode 2045 — Second Minimum Time to Reach Destination (Hard)

Specification

- n nodes, unweighted bidirectional edges. Every edge takes time minutes to cross.
- Traffic signals: a node can only be **entered** during specific windows — if you arrive when the signal is red (an odd multiple of change has passed), you must wait until it turns green, in increments of change minutes.
- Find the **second minimum distinct time** to go from node 1 to node n , including signal wait time.

Example

Input: $n = 5$, $\text{edges} = [[1,2],[1,3],[1,4],[3,4],[4,5]]$, $\text{time} = 3$, $\text{change} = 5$
Output: 13

Algorithm (BFS-based SSSP with two tracked distances per node)

Since all edges have equal weight, this is really **BFS**, not heap-Dijkstra — but it's still "single source shortest path," just adapted to track the *two* smallest distinct path lengths (in edge-count) per node instead of one:

1. BFS from node 1, but keep $\text{dist1}[]$ and $\text{dist2}[]$ (first and second-shortest distinct edge-counts) per node instead of just one $\text{dist}[]$.
2. A neighbor gets pushed to the queue if this path length is strictly less than $\text{dist2}[]$ for that neighbor (i.e. it's new information — either a new shortest or new second-shortest).
3. Once you have $\text{dist2}[n]$ in edge-counts, convert edge-count to real time by simulating the signal wait at each of those $\text{dist2}[n]$ steps.

Complexity: $O(n + m)$ for the BFS, $O(\text{dist2}[n])$ for the time simulation.

Code

```
class Solution {
public:
    int secondMinimum(int n, vector<vector<int>>& edges, int time, int change) {
        vector<vector<int>> adj(n + 1);
        for (auto& e : edges) {
            adj[e[0]].push_back(e[1]);
            adj[e[1]].push_back(e[0]);
        }

        vector<int> dist1(n + 1, -1), dist2(n + 1, -1);
        queue<pair<int,int>> q; // {node, steps}
        dist1[1] = 0;
        q.push({1, 0});

        while (!q.empty()) {
            auto [u, steps] = q.front(); q.pop();
            for (int v : adj[u]) {
                if (dist1[v] == -1) {
                    dist1[v] = steps + 1;
                    q.push({v, steps + 1});
                }
            }
        }
    }
};
```

```

        } else if (dist2[v] == -1 && steps + 1 != dist1[v]) {
            dist2[v] = steps + 1;
            q.push({v, steps + 1});
        }
    }
}

int totalTime = 0;
for (int i = 0; i < dist2[n]; i++) {
    // if currently in a "red" window, wait until the next green
    if ((totalTime / change) % 2 == 1) {
        totalTime = (totalTime / change + 1) * change;
    }
    totalTime += time;
}
return totalTime;
};

```

Summary Table

#	Problem	Core Trick on Top of Base Dijkstra	Difficulty
743	Network Delay Time	Plain Dijkstra, answer = max(dist)	Medium
787	Cheapest Flights Within K Stops	Bellman-Ford, bounded to k+1 rounds	Medium
1631	Path With Minimum Effort	Relax with max instead of + (minimax)	Medium
1514	Path with Maximum Probability	Relax with *, use a max-heap	Medium
1976	Number of Ways to Arrive	Track dist[] + ways[] together	Medium
2045	Second Minimum Time	BFS variant, track 2 smallest distances/node	Hard

Suggested solving order: 743 → 1631 → 1514 → 1976 → 787 → 2045 (roughly increasing distance from the plain Dijkstra base template).

Key exam reminder: if a constraint limits the **number of edges/stops** used (like 787), plain Dijkstra's greedy finalization breaks — that's your signal to switch to bounded Bellman-Ford instead. Every other variation here keeps Dijkstra's heap structure and just changes the relax rule or what's tracked alongside dist[].